



ESTRUTURA DE DADOS I

FUNDAMENTOS E ESTRUTURAS LINEARES

Prof Me Wesley Soares de Souza

✦
👤 Boas-vindas

REFINAMENTO SUCESSIVO

Começar com uma solução geral e detalhá-la progressivamente até que possa ser implementada.

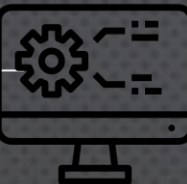
PROBLEMA: CALCULAR A MÉDIA DE TRÊS NOTAS.

NÍVEL 1 — ABSTRAÇÃO

Calcular a média das notas

NÍVEL 2 — REFINAMENTO

1. Obter as três notas
2. Somar as notas
3. Dividir a soma por três
4. Exibir a média



REFINAMENTO SUCESSIVO

- **NÍVEL 3 — MAIS DETALHADO**

1. Ler nota1
2. Ler nota2
3. Ler nota3
4. $\text{soma} \leftarrow \text{nota1} + \text{nota2} + \text{nota3}$
5. $\text{media} \leftarrow \text{soma} / 3$
6. Exibir media

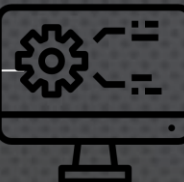
MENSAGEM CENTRAL

Problema complexo

Subproblemas menores

Passos simples

Algoritmo implementável



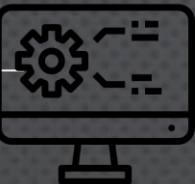
ATIVIDADE RÁPIDA: REFINANDO UM PROBLEMA

PROBLEMA

- DESENVOLVER UM ALGORITMO PARA CONTROLAR UMA FILA DE ATENDIMENTO.

VOCÊS DEVEM IDENTIFICAR:

- QUAIS DADOS PRECISAM SER ARMAZENADOS?
- COMO UMA PESSOA ENTRA NA FILA?
- COMO UMA PESSOA SAI DA FILA?
- QUEM DEVE SER ATENDIDO PRIMEIRO?



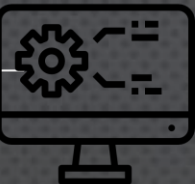
TIPOS ABSTRATOS DE DADOS

UM **TIPO ABSTRATO DE DADOS (TAD)** DEFINE:

- QUAIS DADOS EXISTEM
- QUAIS OPERAÇÕES PODEM SER REALIZADAS
- QUAL O COMPORTAMENTO ESPERADO

SEM DEFINIR NECESSARIAMENTE:

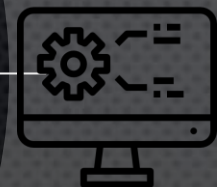
- COMO OS DADOS SERÃO ARMAZENADOS
- COMO AS OPERAÇÕES SERÃO IMPLEMENTADAS



ABSTRAÇÃO E ENCAPSULAMENTO

Pilha

- Empilhar()
- Desempilhar()
- Topo()
- Implementação interna escondida



ABSTRAÇÃO E ENCAPSULAMENTO

Pilha

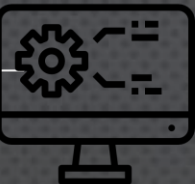
- Empilhar()
- Desempilhar()
- Topo()
- Implementação interna escondida

Interface

O QUE a estrutura faz.

Implementação

COMO a estrutura faz

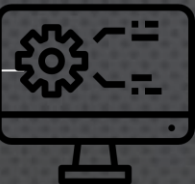


ABSTRAÇÃO E ENCAPSULAMENTO

UMA PILHA PODE SER IMPLEMENTADA UTILIZANDO:

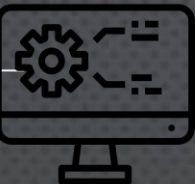
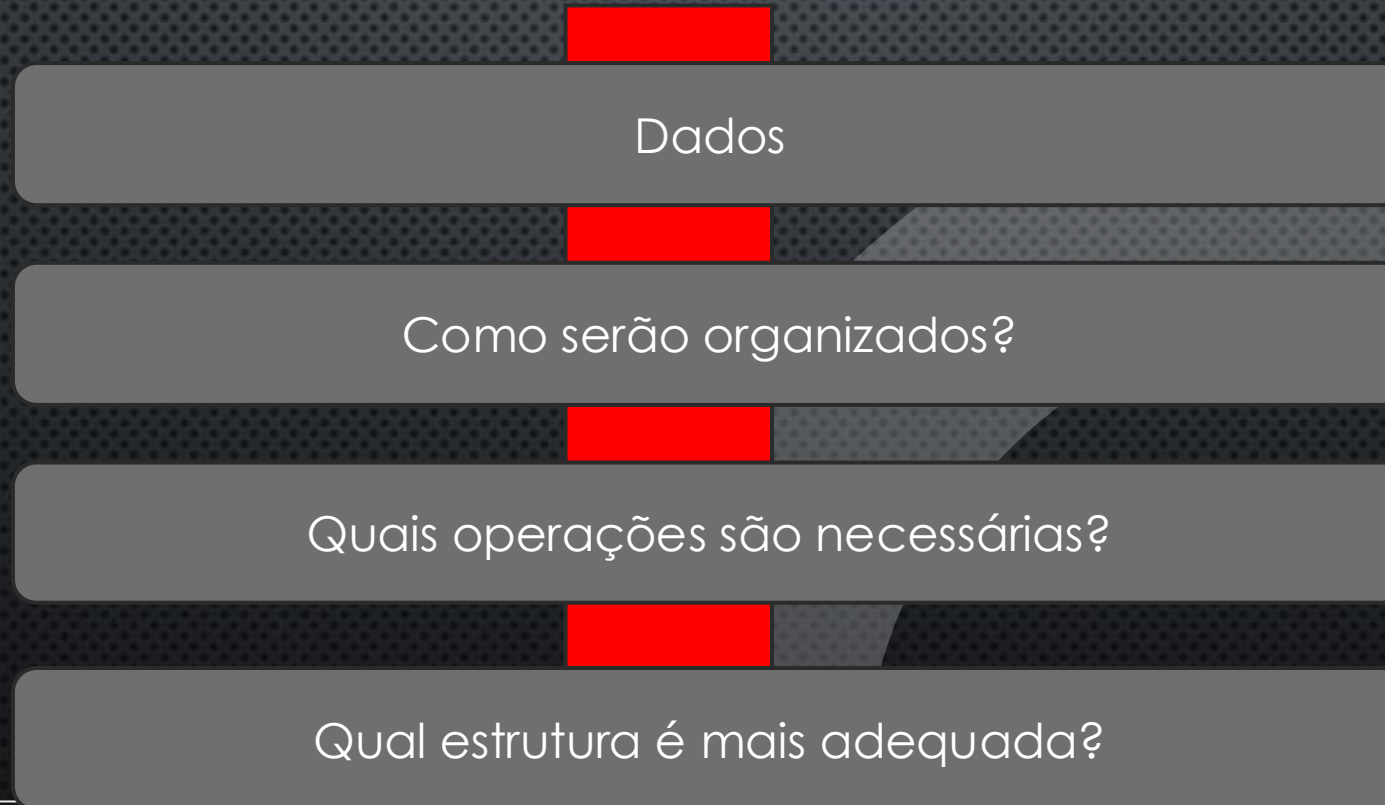
- VETOR
- LISTA LIGADA

MAS O COMPORTAMENTO EXTERNO CONTINUA SENDO O DE UMA PILHA.



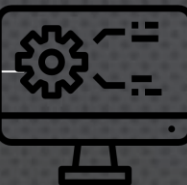
ESTRUTURAS DE DADOS: ESCOLHENDO A REPRESENTAÇÃO

- MESMO PROBLEMA, DIFERENTES ESTRUTURAS



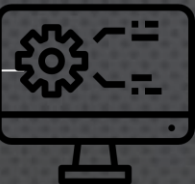
ESTRUTURAS DE DADOS: ESCOLHENDO A REPRESENTAÇÃO

Necessidade	Estrutura possível
Acesso por posição	Lista sequencial
Inserção e remoção frequente	Lista ligada
Último a entrar, primeiro a sair	Pilha
Primeiro a entrar, primeiro a sair	Fila
Relações hierárquicas	Árvore
Relações entre elementos	Grafo



CONCEITOS PRINCIPAIS DESTA AULA

- ALGORITMOS SÃO SEQUÊNCIAS DE PASSOS PARA RESOLVER PROBLEMAS.
- O PROJETO DE ALGORITMOS TRANSFORMA PROBLEMAS EM SOLUÇÕES.
- O REFINAMENTO SUCESSIVO PERMITE DETALHAR SOLUÇÕES COMPLEXAS.
- ESTRUTURAS DE DADOS ORGANIZAM E ARMAZENAM INFORMAÇÕES.
- TADs DEFINEM OPERAÇÕES E COMPORTAMENTOS SEM EXPOR A IMPLEMENTAÇÃO.



OBJETIVO DA AULA

- ENTENDER POR QUE ALGORITMOS PRECISAM SER ANALISADOS;
 - DIFERENCIAR TEMPO DE EXECUÇÃO E QUANTIDADE DE OPERAÇÕES;
 - RELACIONAR O TAMANHO DA ENTRADA AO CUSTO DO ALGORITMO;
 - IDENTIFICAR MELHOR, PIOR E CASO MÉDIO;
 - RECONHECER AS PRINCIPAIS CLASSES DE COMPLEXIDADE;
 - INTERPRETAR E UTILIZAR A NOTAÇÃO O ;
 - COMPREENDER A IDEIA DA NOTAÇÃO Ω ;
-
- COMPARAR ALGORITMOS QUE RESOLVEM O MESMO PROBLEMA.

ANÁLISE DE ALGORITMOS

“Dois algoritmos podem resolver o mesmo problema, mas não necessariamente com o mesmo custo.”



POR QUE ANALISAR ALGORITMOS?

- IMAGINE DOIS ALGORITMOS PARA PESQUISAR UM ELEMENTO EM UMA LISTA.

Algoritmo A

Verifica os elementos um por um.

Algoritmo B

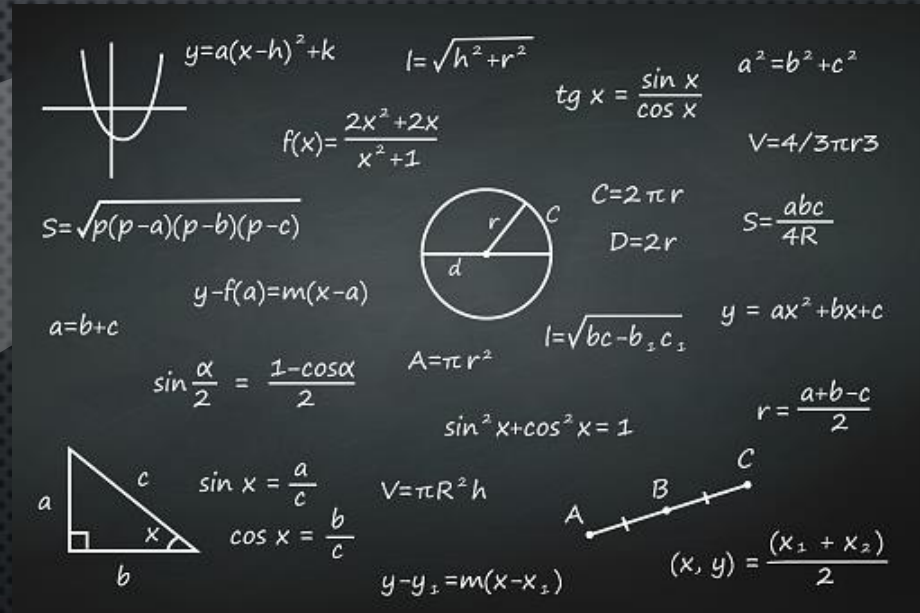
Consegue eliminar metade dos elementos a cada comparação.

Para uma lista pequena, talvez a diferença seja imperceptível.

Para **1.000.000 de elementos**, a diferença pode ser enorme.

QUESTÕES QUE QUEREMOS RESPONDER

- QUANTO TRABALHO O ALGORITMO REALIZA?
- COMO ESSE TRABALHO CRESCE?
- QUAL ALGORITMO É MAIS EFICIENTE?
- O QUE ACONTECE QUANDO A ENTRADA AUMENTA?



O TAMANHO DA ENTRADA IMPORTA

CONCEITO

- CHAMAMOS DE **TAMANHO DA ENTRADA** A QUANTIDADE DE DADOS QUE O ALGORITMO PRECISA PROCESSAR.
- REPRESENTAMOS NORMALMENTE POR: **N**

Entrada	N
Somar 10 números	$n = 10$
Pesquisar em 1.000 registros	$n = 1.000$
Ordenar 50.000 elementos	$n = 50.000$

PERGUNTA



Se dobrarmos o tamanho da entrada, o trabalho do algoritmo também dobra?

DO PROBLEMA À SOLUÇÃO

Algoritmo:

1. Ler n
2. Repetir n vezes:
 <executar uma operação>
3. Finalizar

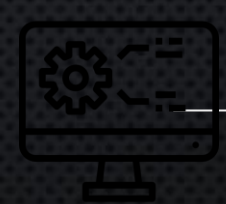
Quantas vezes será a execução?

$$N = 10$$

$$N = 100$$

$$N = 1.000$$

O custo cresce proporcionalmente a **n** .



NÃO QUEREMOS DEPENDER DO COMPUTADOR

Uma abordagem inicial seria medir:

- “Meu algoritmo executou em 0,5 segundos.”

O tempo depende de:

- Processador
- Memória
- Linguagem
- Compilador
- Sistema operacional
- Implementação
- Dados de entrada



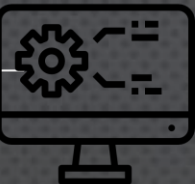
NÃO QUEREMOS DEPENDER DO COMPUTADOR

EM VEZ DE PERGUNTAR:

- **QUANTO TEMPO O PROGRAMA LEVOU?**

PERGUNTAMOS:

- **QUANTAS OPERAÇÕES O ALGORITMO PRECISA REALIZAR À MEDIDA QUE A ENTRADA CRESCE?**



CUSTO COMPUTACIONAL

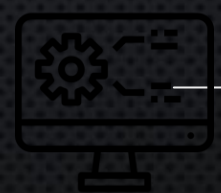
O **custo computacional** representa os recursos necessários para executar um algoritmo.

Tempo

- Quantidade de operações necessárias.

Espaço

- Quantidade de memória utilizada.



COMPLEXIDADE TEMPORAL



CONTANDO OPERAÇÕES

```
para i = 1 até n  
    escreva(i)  
fim
```

Quantas vezes escreva(i)
será executado?

n vezes

Portanto:

- $T(n)=n$

Se $n = 10$ (10 operações)

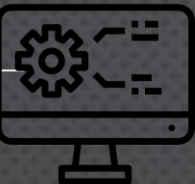
O crescimento é **linear**.

E QUANDO TEMOS DUAS OPERAÇÕES?

```
para i = 1 até n  
  escreva(i)  
fim
```

```
para j = 1 até n  
  escreva(j)  
fim
```

- **Primeiro laço:** n
- **Segundo laço:** n
- **Total:** $T(n) = n + n$
 $T(n) = 2n$



ANÁLISE ASSINTÓTICA

- MAS PRECISAMOS REALMENTE NOS PREOCUPAR COM O 2 ?

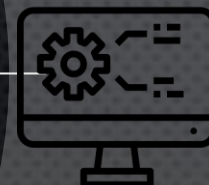
PARA ANÁLISE DE CRESCIMENTO, NORMALMENTE CONSIDERAMOS:

- $T(N) = 2N$

COMO:

- $O(N)$

Na análise assintótica, estamos interessados principalmente em como o custo cresce quando n aumenta.



MELHOR CASO, PIOR CASO E CASO MÉDIO

CONSIDERE UMA PESQUISA SEQUENCIAL:

[10, 25, 31, 42, 58, 70, 81]

QUEREMOS ENCONTRAR: 10

81

<ELEMENTO QUALQUER>

Melhor caso

1 comparação

Pior caso

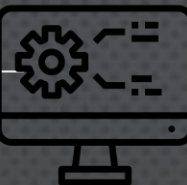
n comparação

Caso médio

Depende da
distribuição das
posições.

MELHOR CASO, PIOR CASO E CASO MÉDIO

Caso	Custo
Melhor	menor quantidade de operações
Médio	comportamento esperado
Pior	maior quantidade de operações



O QUE É COMPLEXIDADE ASSINTÓTICA?

Como o custo de um algoritmo cresce quando o tamanho da entrada aumenta.

- **Não estamos interessados em:**
 - Processador específico;
 - Tempo exato em segundos;
 - Pequenas diferenças constantes.
- **Estamos interessados no:**
 - **comportamento de crescimento.**

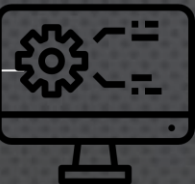


O QUE É COMPLEXIDADE ASSINTÓTICA?

$$T(n) = 3n + 10$$

- Para entradas muito grandes: $3N + 10$
- É dominado pelo termo: N

Complexidade $O(n)$



NOTAÇÃO BIG O — $O(N)$

BIG O INDICA UMA ORDEM DE CRESCIMENTO QUE O ALGORITMO NÃO ULTRAPASSA ASSINTOTICAMENTE, IGNORANDO CONSTANTES E TERMOS DE MENOR ORDEM.

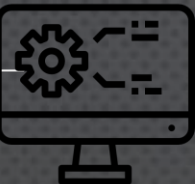
$$T(n) = 5n$$

$$T(n) = 3n + 20$$

$$T(n) = 100n + 50$$

Crescimento $O(n)$

Complexidade linear



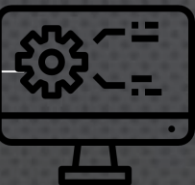
PRINCIPAIS CLASSES DE COMPLEXIDADE

Mais eficiente



Menos eficiente

Nome	Tipo
$O(1)$	Constante
$O(\log n)$	Logarítmica
$O(n)$	Linear
$O(n \log n)$	Linear-logarítmica
$O(n^2)$	Quadrática
$O(n^3)$	Cúbica
$O(2^n)$	Exponencial
$O(n!)$	Fatorial



$O(1)$: COMPLEXIDADE CONSTANTE

```
x = vetor[5]
```

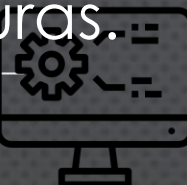
Independentemente do tamanho do vetor, acessamos diretamente uma posição.

Custo

- $O(1)$

Exemplos

- Acessar uma posição de um vetor;
- Consultar uma variável;
- Inserir/remover em uma posição conhecida de determinadas estruturas.

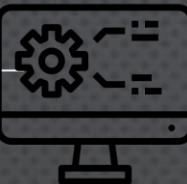


$O(1)$: COMPLEXIDADE CONSTANTE

N aumenta



custo permanece aproximadamente igual



$O(\log N)$: COMPLEXIDADE LOGARÍTMICA

- IMAGINE PROCURAR UM NOME EM UMA LISTA ORDENADA.
- EM VEZ DE VERIFICAR UM ELEMENTO POR VEZ:

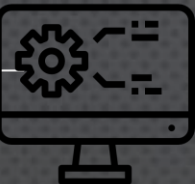
1.000.000 elementos

Verifica o meio

Verifica metade

...

A cada passo, o problema é reduzido aproximadamente à metade.



$O(\log N)$: COMPLEXIDADE LOGARÍTMICA

- IMAGINE PROCURAR UM NOME EM UMA LISTA ORDENADA.
- EM VEZ DE VERIFICAR UM ELEMENTO POR VEZ:

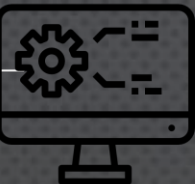
1.000.000 elementos

Verifica o meio

Verifica metade

...

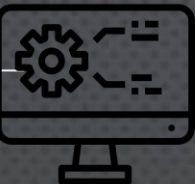
A cada passo, o problema é reduzido aproximadamente à metade.



$O(N)$: COMPLEXIDADE LINEAR

para cada elemento
verificar elemento
fim

- 10 elementos $\rightarrow$ aproximadamente 10 operações
- 100 elementos $\rightarrow$ aproximadamente 100 operações
- Pesquisa sequencial.

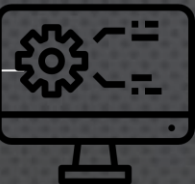


$O(N^2)$: COMPLEXIDADE QUADRÁTICA

```
para i = 1 até n
  para J = 1 até n
    operação
  fim
fim
```

- $n \times n = n^2$

- Alguns algoritmos simples de ordenação, como **Bubble Sort** e **Selection Sort**, possuem comportamento quadrático em seus casos típicos/pior caso.



COMPARANDO O CRESCIMENTO

n	$O(1)$	$O(\log n)$	$O(n)$	$O(n^2)$
10	1	~ 3	10	100
100	1	~ 7	100	10.000
1000	1	~ 10	1000	1.000.000
10.000	1	~ 13	10.000	100.000.000

A diferença entre algoritmos torna-se cada vez mais importante conforme o tamanho da entrada aumenta.

ATIVIDADE PRÁTICA: QUAL É A COMPLEXIDADE?

- `X = VETOR[10]`

$O(1)$

ATIVIDADE PRÁTICA: QUAL É A COMPLEXIDADE?

- PARA $i = 1$

ATÉ N ESCREVA(i)

FIM

$O(n)$

ATIVIDADE PRÁTICA: QUAL É A COMPLEXIDADE?

- PARA I = 1 ATÉ N
 PARA J = 1 ATÉ N
 ESCREVA(I, J)
 FIM
FIM

$O(n^2)$

ATIVIDADE PRÁTICA: QUAL É A COMPLEXIDADE?

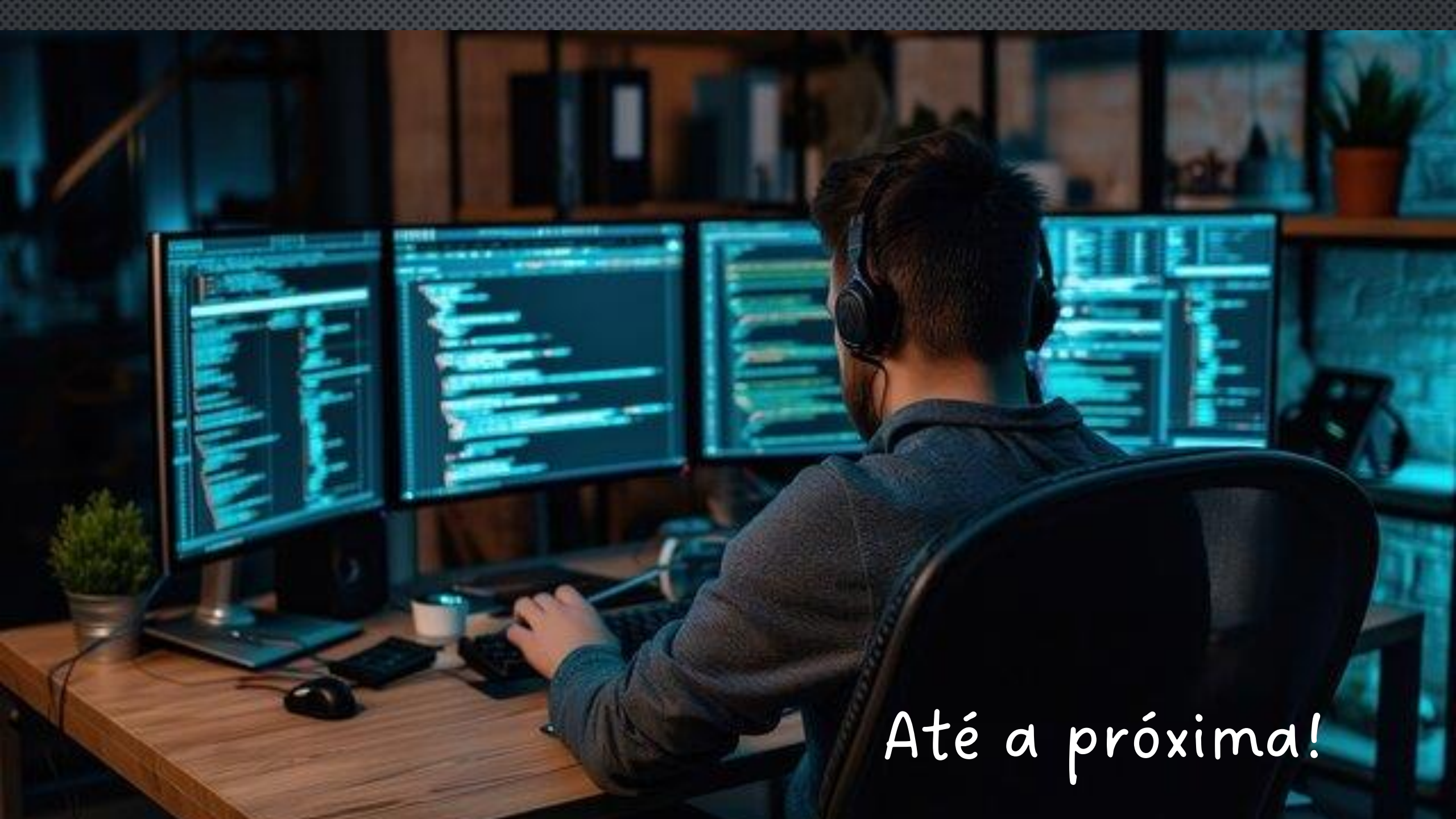
- O QUE ACONTECERIA SE O SEGUNDO LAÇO FOSSE EXECUTADO APENAS METADE DAS VEZES?



FECHAMENTO

SE DOIS ALGORITMOS RESOLVEM O MESMO PROBLEMA, POR QUE ESCOLHER UM EM VEZ DO OUTRO?

UM **ALGORITMO CORRETO** RESOLVE O PROBLEMA. UM **ALGORITMO EFICIENTE** RESOLVE O PROBLEMA USANDO OS RECURSOS DE FORMA ADEQUADA.



Até a próxima!